

TrendMiner Solutions Training — OEE Reporting (Hands-On Notebook)

Date: 2025-09-30 (Europe) / 2025-10-08 (Americas)

Audience: Experienced TrendMiner users

Goal: Build an end-to-end OEE KPI reporting workflow using TrendMiner.

Table of Contents

- [Solution training overview](#)
- [OEE overview](#)
- [Prerequisites](#)
- [Workshop steps at a glance](#)
- [Create context types & context fields](#)
- [Create context items in TrendMiner with value-based search](#)
- [Create a monitor to automatically generate new context items](#)
- [TrendMiner SDK Setup](#)
- [Load and write context items](#)
- [Visualize and validate results](#)
- [Create custom visualizations](#)
- [Build dashboard](#)

Tip: In **JupyterLab**, you can also enable the built-in **Table of Contents** panel from the left sidebar (bullet-list icon). If you don't see it, check **View → Left Sidebar → Show Table of Contents**.

Solution training overview

Welcome to the **TrendMiner Solutions Training** workshop!

This session is designed for experienced TrendMiner users who want to learn how to **build custom solution packages** directly on the TrendMiner platform. Rather than being a theoretical exercise, this workshop mirrors the approach used by TrendMiner Data Analytics Engineers when implementing real-world solutions for customers.

By the end of this training, you will have learned how to:

- Capture production events (batches for batch processes, operating periods for continuous processes) with monitors in TrendHub and display them as context items in a ContextHub view
- Load those context items via Python and write loss data to their context fields using the TrendMiner SDK
- Capture regular periods (weeks/months) and compute OEE KPIs (availability, productivity, quality, overall OEE) over those periods, using the data of production events captured earlier
- Bring enriched context items plus custom visuals into actionable user dashboards

Although the outcome is a working OEE reporting solution, the real value is learning a **general solutions approach** that you can reuse for many other applications (e.g., energy reporting, batch validation, or quality monitoring). This workshop mirrors how a TrendMiner Data Analytics Engineer implements custom solutions for customers — with practical tips, tricks, and best practices.

OEE overview

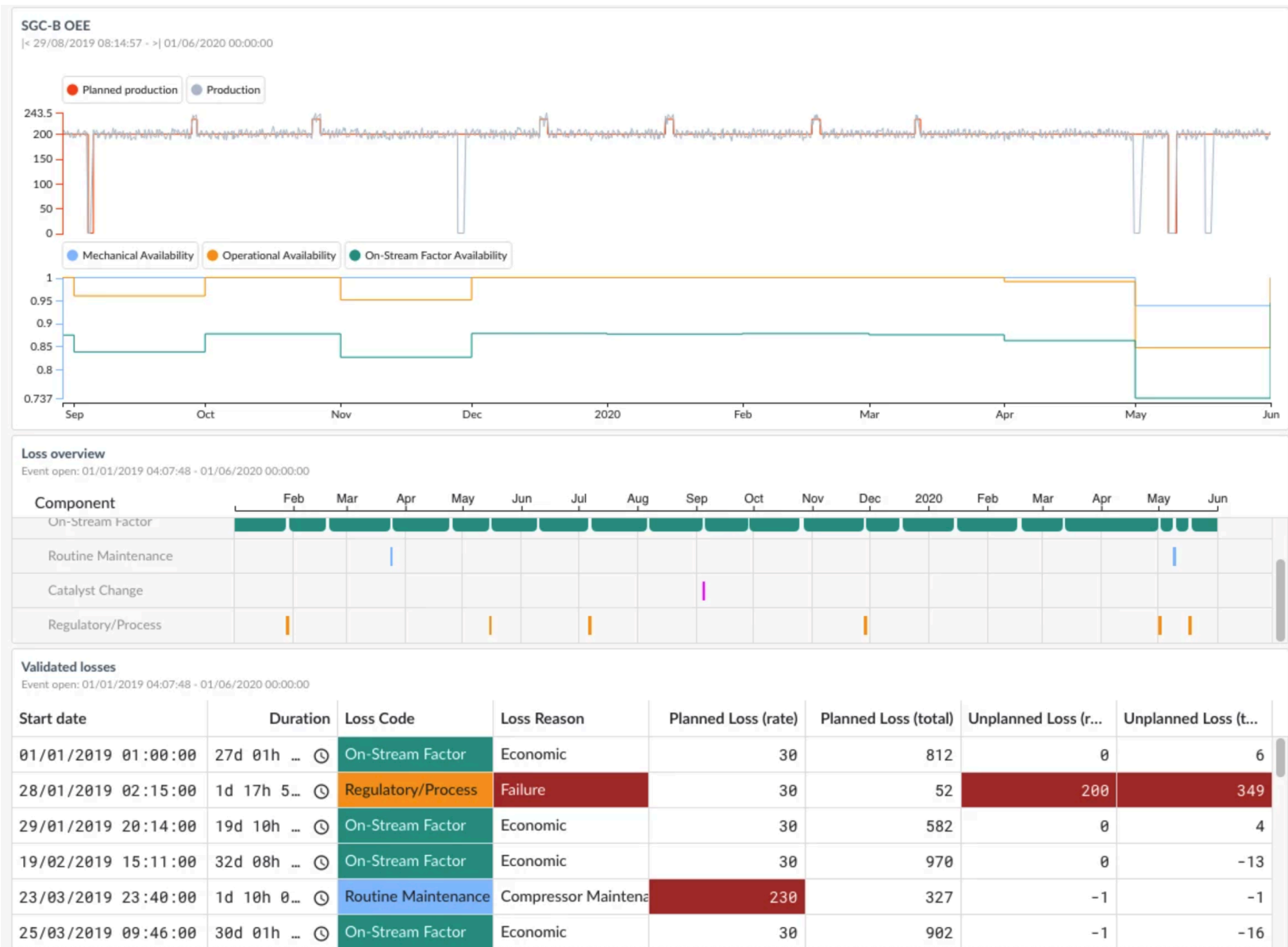
Overall Equipment Effectiveness (OEE) is the gold standard for measuring manufacturing productivity. Simply put, it identifies the percentage of manufacturing time that is truly productive.

An OEE score of 100% means you are manufacturing only good parts, as fast as possible, with no stop time. In OEE terms, that corresponds to:

- 100% Quality (only good parts)
- 100% Performance (as fast as possible)
- 100% Availability (no stop time)

Why TrendMiner for OEE?

TrendMiner provides a highly flexible approach to setting up OEE tracking systems, tightly connecting OEE metrics with production data for intuitive, two-way operations. Engineers can efficiently identify and categorize losses by analyzing trend data and leveraging analytics to calculate integral values promptly. Once collected, OEE data becomes a powerful resource for building reporting flows and performing retrospective analyses to highlight major losses and improvement opportunities.



Prerequisites

- Access to your TrendMiner environment (URL, client credentials, and user credentials)

- Python with the TrendMiner SDK installed (wheel or pip package provided by TrendMiner)
- TrendMiner admin rights to create Context Types & Context Fields in ContextHub settings
- A process (batch or continuous) with tags you can read

Workshop steps at a glance

1. Create custom context types & context fields for production events and regular reporting period (**ContextHub**)
2. Capture production events (batch or operating periods) as context items using value-based search (**TrendHub**)
3. Create a monitor to capture future events as context items (**TrendHub**)
4. Create context items for the regular reporting period via value-based search and monitor (**TrendHub**)
5. Visualize newly created context items in context view (**ContextHub**)
6. Load context items into Python and compute losses per event (**Python SDK**)
7. Compute OEE metrics per regular reporting period using event-level losses (**Python SDK**)
8. Write results back to context fields and update context items (**Python SDK**)
9. Visualize & validate results (**ContextHub**)
10. Create custom visualizations (**MLHub**)
11. Build dashboard (**DashHub**)

Create context types and context fields (ContextHub)

We will create two context types - one for the individual production events and one for the regular reporting period. Each context type will have their own context fields.

A. Individual Production Events (ex: Context Type - OEE Batch)

Fields:

- OEE Quality Loss
- OEE Performance Loss

B. Regular Reporting Period (ex: Context Type - OEE Monthly)

Fields:

- OEE Availability Score
- OEE Performance Score
- OEE Quality Score
- OEE Score

Steps

1. Navigate to ContextHub settings.
2. Create the context fields needed for the context types.
 - Under Platform Configuration, navigate to Context fields.
 - To create the new field, click the **Add field** button at the top right.
 - Provide a Technical name (used in the SDK), Name, and Field type (**Number**).
 - Do this for each of the 6 fields.
3. Create the 2 context types.
 - Under Platform Configuration, navigate to Context item types.
 - To create the new context type, click the **Add item** button at the top right.
 - Provide a Name and specify the color & icon
 - Add the relevant context fields
 - For the **Regular Reporting Period** context type, toggle on the Set field value as label for the OEE Score context field

Note: Admin rights are necessary to access the settings page.

The screenshot shows the ContextHub interface with a sidebar on the left containing sections like PLATFORM CONFIGURATION, DIGITAL TWIN MANAGER, and SCHEDULE MANAGER. The main area is titled 'Context item types' and features a search bar with the text 'oee month'. Below the search bar is a table with the following data:

Source	Technical identifier	Name
Polymerization OEE	d4399f1827284402a29343e97a7d0d12	OEE Monthly Report

Below the table, there is a pagination control showing '1 of 1'. On the right side, a modal window titled 'Configure OEE Monthly Report' is open. It contains several sections: 'TYPE DEFINITION' with a 'Technical identifier' field, a 'Name' field, and a 'Workflow' dropdown; 'VISUALIZATION' with a 'Color' field and an 'Icon' dropdown; 'CONTEXT FIELDS' with four rows for 'Availability (OEE)', 'OEE score (OEE)', 'Performance (OEE)', and 'Quality (OEE)', each with a dropdown and a toggle switch; and 'OPTIONS' with two checkboxes: 'Approvals available' and 'Turn on history'. At the bottom of the modal are 'Cancel' and 'Save' buttons.

Create context items in TrendMiner with value-based search (TrendHub)

In this step, you will create context items in the TrendMiner using **value based search**.

We'll create **two sets** of context items:

1. **Production events** (batches or continuous operating periods) — Context Type: `OEE_Batch`
2. **Reporting periods** (ex: monthly) — Context Type: `OEE_Monthly`

A) Production events via value based search

Goal: Identify each batch (or operating window) with precise start and end times, then save as context items.

Note: Accurately capturing the event boundaries will help ensure reliable OEE metrics.

1. **Choose the key process tags** that reliably indicate event boundaries. Common indicators:

- **Batch/State tag:** step tag that directly records (ex: `BATCH_ACTIVE` from 0→1 at start, 1→0 at end).
- **Feed/Flow tag:** above threshold → “running”, below threshold or near-zero → “not running”.
- **Equipment status:** on/off transitions to bracket operation.
- **Phase/Step index:** changes delineate batch phases (use combined logic for full batch bounds).

2. **Build the value based search** (examples):

- **State-based:** Find events where `BATCH_ACTIVE = 1` (start) and `= 0` (end).
- **Threshold-based:** Find periods where `FEED_FLOW > X` (minimum duration X minutes).
- **Composite logic:** `RUN_STATUS == 1 AND FEED_FLOW > X` to reduce false positives.

3. **Validate** the results by visualizing on the Focus Chart:

- Check that the search results align with your expectations.
- Adjust minimum duration to avoid noise or outliers.
- Add filters (ex: **Unit**, **Product**) if needed.

4. **Add as context items:**

- Select the options menu next to results and click `Add as context item`
- Designate which search results you want as context items and click `Add`
- Select both Component and Context Type `OEE_Batch` and click `Save` .

5. **Save the value based search**

- Save the value based search
- Enabling the monitor requires a saved value based search

Tip: For continuous processes, define **operating periods** similarly.

B) Reporting periods via value based search (ex: monthly)

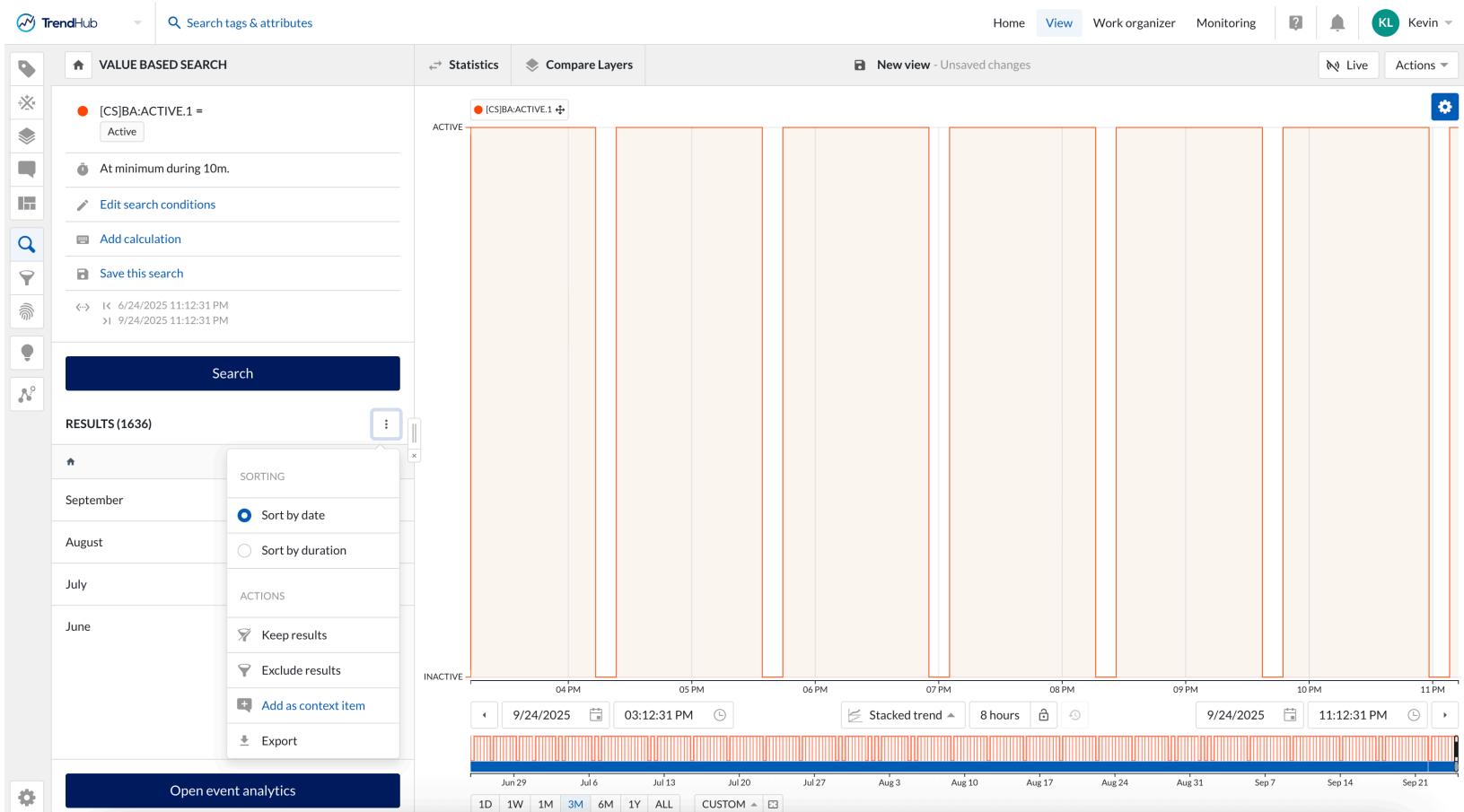
Goal: Create context items that bucket time for KPI rollups and reporting (Availability, Performance, Quality, OEE Score).

- Use the **time tag** `TM_month_timezone` to identify **monthly** boundaries in your working timezone.
- Build a value based search that finds **each month's range** (start at the first instant of the month to the last).
- Save results as Context Items with Context Type `OEE_Monthly` .

Other options: Yearly (`TM_year_timezone`), daily (`TM_day_timezone`), or your own time tags for custom reporting periods.

Best practices

- Keep the value based search simple and interpretable; document chosen tags and thresholds.
- Prefer edge/threshold logic that is robust to noise and sensor dropouts.
- Spot check several items across different operating modes/products before moving on.



Add search results as context items.

The screenshot shows the TrendHub interface. On the left, a sidebar contains a search bar and a list of months with their respective counts: September (425), August (551), July (552), and June (108). The main area displays a search for '[CS]BA:ACTIVE.1' with a 'VALUE BASED SEARCH' filter. A modal titled 'Add context items' is open on the right, showing fields for Component, Type, Description, Keywords, and various OEE metrics. The 'Type' field is set to 'OEE Poly Batch'. The 'Save 1,636 items' button is visible at the bottom right of the modal.

Select the correct tag/component and context type (OEE_Batch).

Create a monitor to automatically generate new context items (TrendHub)

In the previous step, we created context items for all of our past production events. Now, we will enable monitors to automate the generation of future events as context items.

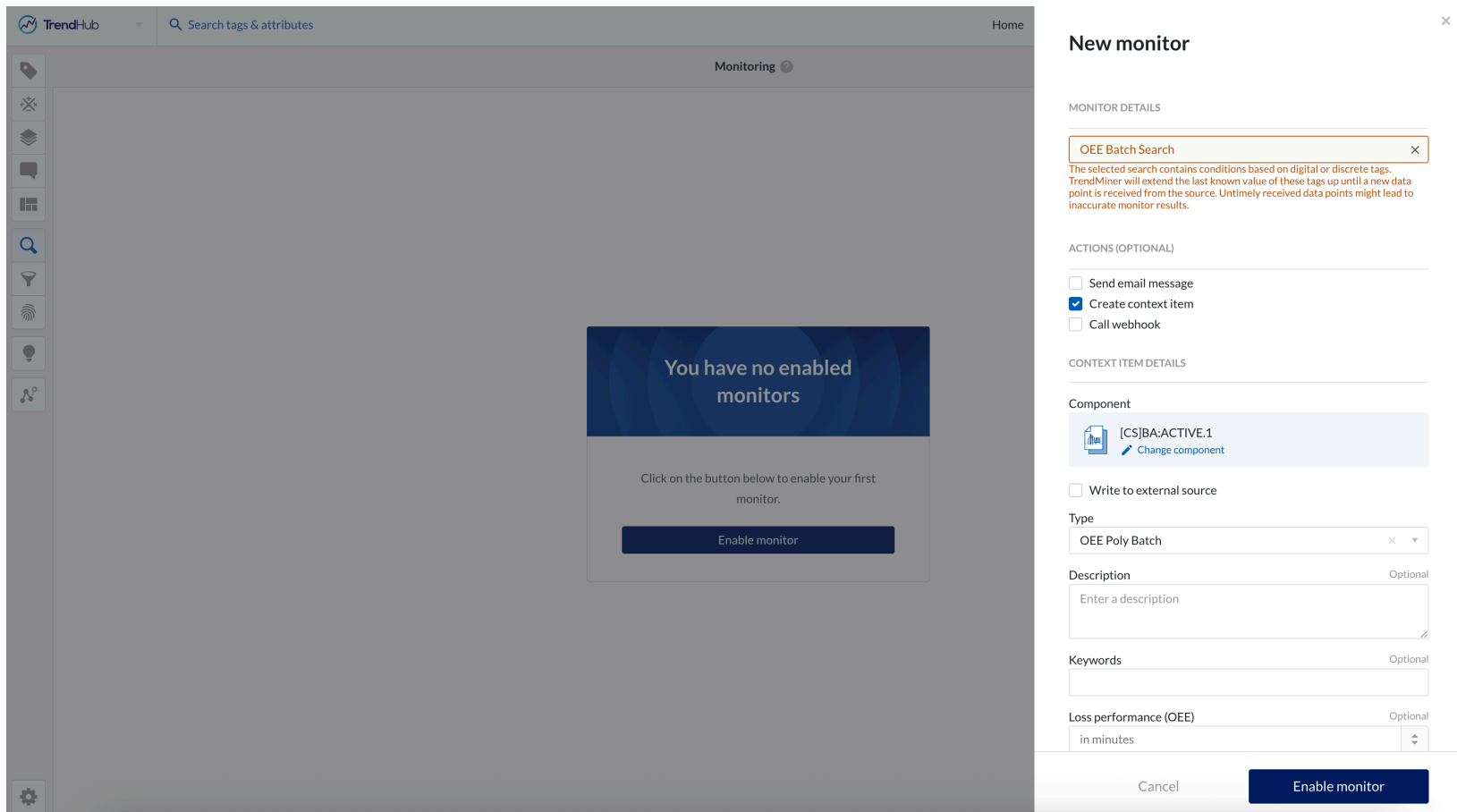
Create two monitors:

1. **Production event monitor** — captures batches/operating periods as `OEE_Batch` context items.

2. **Reporting period monitor** — captures regular time buckets (ex: month) as **OEE_Monthly** context items.

Steps:

1. Navigate to **Monitoring** and click **Enable monitor**.
2. Select the searches that we created from the previous step.
3. Select **Create context item** as monitor action.
4. Specify the Component and Type.
5. Click **Enable monitor**.



Enable monitor - select the same tag/component and context type as previous step.

TrendMiner SDK Setup (Python SDK)

Install the TrendMiner SDK

The latest SDK version is available on GitHub: <https://github.com/TrendMinerCS/sdk>

To install directly from the provided wheel (recommended for this workshop), run:

Note: the below code may not be the latest SDK version, depending on your time of viewing this document.
Please check the GitHub link above to ensure that you download the most up-to-date SDK.

```
In [ ]: # Uncomment the next line to install:  
# !pip install https://github.com/TrendMinerCS/sdk/raw/main/sdk/trendminer_interface-0.1.0.post158-py3-none-any.whl  
# !pip install https://github.com/TrendMinerCS/sdk/raw/main/sdk/trendminer_interface-0.1.0.post172-py3-none-any.whl
```

Connect to TrendMiner

A client needs to be created in order to use the SDK. To create a client, navigate to **ConfigHub → Security → Clients → Add client**.

You will need the client ID and client secret in the next step.

ConfigHub

?

KL Kevin

DASHBOARD

Services

SETTINGS

Email configuration

Home page configuration

DOCUMENTS

Work organizer

SECURITY

Users

Groups

Clients

Access management

Certificates

Identity providers

DATA

Settings

Connectors

Add client

Client overview

Client ID
aglushchenkoclient
awelsingclient
chfclient
client
cnr
dmerschhclient
dmunchrclient
eduardoclient
fmotteclient
fvandaelclient
igorclient
Jordiclient
jorisclient
kevinliclient
mlepperclient
rebasclient

1

of 2

Next →

RECENT ACTIONS

restart-edge-manager	8/22/2025 1:57:02 AM	SUCCESS
configure-tm-platform	8/22/2025 1:39:23 AM	SUCCESS
configure-tm-agent	8/22/2025 1:39:21 AM	SUCCESS
restart-edge-manager	8/22/2025 1:22:15 AM	SUCCESS
configure-tm-platform	8/22/2025 1:03:00 AM	SUCCESS
configure-tm-agent	8/22/2025 1:02:50 AM	SUCCESS
configure-smtp	8/21/2025 9:45:06 AM	SUCCESS
configure-smtp	8/21/2025 9:44:07 AM	SUCCESS
restart-edge-manager	8/12/2025 2:55:39 AM	SUCCESS
configure-tm-platform	8/12/2025 2:26:56 AM	SUCCESS
configure-tm-agent	8/12/2025 2:26:49 AM	SUCCESS
install-updates	8/12/2025 2:26:42 AM	SUCCESS
install-updates	8/12/2025 2:26:36 AM	SUCCESS

Create client if you do not have one already.

Specify the correct timezone for you.

Common pytz time zones:

- America/New_York
- America/Chicago
- America/Denver
- America/Los_Angeles

```
In [ ]: from trendminer_interface import TrendMinerClient
import keyring

url = "https://cs.trendminer.net"
client_id = "kevinliclient"
username = "keli"

client_secret = keyring.get_password(url, client_id)
password = keyring.get_password(url, username)

client = TrendMinerClient(url=url,
                          client_id=client_id,
                          client_secret=client_secret,
                          username=username,
                          password=password,
                          tz="America/Chicago",
                          verify=False)

print(client.version)
```

The print statement outputs the TrendMiner version and confirms that the client has been successfully created.

Load and write context items (Python SDK)

Below are code blocks for loading context items, calculating OEE metrics, and writing results to the context fields.

Define our ContextHub view parameters

In this step, we configure all the objects and thresholds that we'll need to calculate OEE losses for each batch:

Batch type → Specifies the context type (`OEE Poly Batch`) we're working with.

Performance & quality fields → The custom fields in ContextHub where performance and quality losses will be written.

Batch component → Tag (`[CS]BA:ACTIVE.1`) that indicates when the batch is active.

User → Context items created by the user `keli` .

Target batch duration → Expected runtime (1 hour), used as a benchmark for performance loss.

Concentration & threshold → Tag (`[CS]BA:CONC.1`) and numeric limit (40) that define whether the batch passes quality.

This setup ensures all subsequent code knows where to read inputs and where to store calculated losses.

```
In [ ]: # Import pandas library – we will store our context items in pandas dataframes
import pandas as pd

# --- Set parameters for batch context analysis ---

# Context type: defines which kind of context items we are working with
batch_type = client.context.type.from_name("OEE Poly Batch")

# Fields in ContextHub where we will later store loss values
batch_performance_field = client.context.field.from_key("OEE_loss_performance")
batch_quality_field = client.context.field.from_key("OEE_quality_loss")

# Component tag representing batch activity (used to link context items to the process)
batch_component = client.tag.from_name("[CS]BA:ACTIVE.1")

# Only consider context items created by this user
user = client.user.from_name("keli")

# Expected batch duration (used as benchmark for performance)
target_batch_duration = pd.Timedelta(hours=1)

# Tag measuring final product concentration, used for quality checks
concentration = client.tag.from_name("[CS]BA:CONC.1")

# Threshold for concentration: batches below this fail the quality check
concentration_threshold = 40
```

Retrieve context items using filters

Here we query ContextHub to get a list of batches that are finished but not yet annotated with losses:

Filters applied:

- Only include context items of type `OEE Poly Batch`.
- Only for the `[CS]BA:ACTIVE.1` component.
- Only created by the user `keli`.
- Only those with an empty performance field (not yet annotated).
- Only closed (finished) batches.

The result is a clean set of finished, unprocessed batches that we can analyze and enrich with loss metrics.

```
In [ ]: # --- Retrieve batches that need annotation ---

chv_batch = client.context.view(
    name="batches",
    filters=[
        # Only include context items of the correct type
        client.context.filter.context_types([batch_type]),

        # Only items associated with the specified batch component tag
        client.context.filter.components([batch_component]),

        # Only context items created by this user
        client.context.filter.users([user]),

        # Only batches where performance field is empty (not annotated yet)
        client.context.filter.field(batch_performance_field, mode="EMPTY"),

        # Only closed (finished) batches are considered
        client.context.filter.states(mode="CLOSED"),
    ]
)

# Retrieve the actual context items matching the filters
batches = chv_batch.get_items()
```

Compute losses for production events

In this step, we calculate performance loss and quality loss for each batch and then update the corresponding ContextHub items:

- Performance loss: Difference between the actual batch duration and the expected (target) batch duration, converted to minutes.
- Quality loss: Based on whether the final concentration meets the threshold. If the product fails (final concentration < threshold), we assign the full batch duration as lost time.
- Update: Results are written back to the batch context items for later use in OEE rollups.

```
In [ ]: # Calculate performance loss in minutes and round to two digits
# Difference between actual batch duration and target duration
batches[batch_performance_field.key] = (
    (batches.index.length - target_batch_duration).total_seconds() / 60
).round(2)

# Calculate quality loss
# Step 1: Get final concentration value for each batch
batches = batches.interval.calculate(tag=concentration, operation="end", name="final concentration")

# Step 2: Determine if quality passed (True if final concentration < threshold → fail)
batches["quality passed"] = batches.pop("final concentration") < concentration_threshold

# Step 3: Assign quality loss in minutes (entire batch duration lost if failed)
batches[batch_quality_field.key] = (
    target_batch_duration.total_seconds() / 60 * batches.pop("quality passed")
)

# Write back updated performance and quality losses to context items
batches.context.update()
```

Compute OEE for regular periods

Here we calculate OEE metrics (Availability, Performance, and Quality) and roll up to the reporting period (monthly OEE KPIs):

- Availability (A): Fraction of the calendar month covered by batch runtime.

- Performance (P): How close actual runtime is compared to the target runtime (after subtracting performance losses).
- Quality (Q): Percentage of runtime that produced good product (after subtracting quality losses).
- OEE: Combined overall score calculated as $OEE = A \times P \times Q$.

We retrieve monthly context items (reporting periods) and all batches within those months. See code below.

```
In [ ]: # --- Setup context types, fields, and tags ---
        oee_type = client.context.type.from_name("OEE Monthly Report")
        availability_field = client.context.field.from_key("OEE_availability")
        performance_field = client.context.field.from_key("OEE_performance")
        quality_field = client.context.field.from_key("OEE_quality")
        oee_total_field = client.context.field.from_key("OEE_oe_score")

        batch_component = client.tag.from_name("[CS]BA:ACTIVE.1")
        batch_type = client.context.type.from_name("OEE Poly Batch")
        batch_performance_field = client.context.field.from_key("OEE_loss_performance")
        batch_quality_field = client.context.field.from_key("OEE_quality_loss")

        component = client.tag.from_name("TM_month_US_Central")
        user = client.user.from_name("keli")

        # --- Retrieve unannotated months (reporting periods) ---
        chv_months = client.context.view(
            name="months",
            filters=[
                client.context.filter.context_types([oee_type]),
                client.context.filter.components([component]),
                client.context.filter.users([user]),
                client.context.filter.field(performance_field, mode="EMPTY"), # only months not yet annotated
                client.context.filter.states(mode="CLOSED"), # only finished months
            ]
        )
        months = chv_months.get_items()

        # --- Retrieve all batches that fall into those months ---
        chv_batch = client.context.view(
            name="batches",
            filters=[
                client.context.filter.context_types([batch_type]),
```

```

        client.context.filter.components([batch_component]),
        client.context.filter.users([user]),
        client.context.filter.interval(months.interval.get_span()) # restrict to month intervals
    ]
)
batches = chv_batch.get_items()
print(batches)

```

Then we compute and update OEE metrics back to ContextHub. See code below.

There is some logic to check for batches that overlap reporting periods (batches that take place between one reporting period and the next). In these overlap cases, the code finds the proportion of the batch in the specific reporting period and weights the OEE metrics by that proportion.

```

In [ ]: # --- Calculate OEE per month ---
for interval in months.index:

    # Find the portion of each batch that overlaps with this month (fraction 0-1)
    in_month_left = batches.index.left.where(batches.index.left > interval.left, interval.left)
    in_month_right = batches.index.right.where(batches.index.right < interval.right, interval.right)
    in_month_right = in_month_right.where(in_month_right > in_month_left, in_month_left)
    in_month_intervals = pd.IntervalIndex.from_arrays(left=in_month_left, right=in_month_right, closed='bo')
    in_month_ratio = in_month_intervals.length / batches.index.length

    # Total batch duration that falls inside this month
    total_batch_duration = (in_month_ratio * batches.index.length).sum(skipna=False)

    # Skip if no batches fall into this month
    if not total_batch_duration:
        continue

    # --- Availability ---
    months.loc[interval, availability_field.key] = round(total_batch_duration / interval.length, 4)

    # --- Performance ---
    batch_performance_losses = pd.to_timedelta(batches[batch_performance_field.key], unit='m')
    total_performance_loss = (in_month_ratio * batch_performance_losses).sum(skipna=False)
    target_total_duration = total_batch_duration - total_performance_loss
    months.loc[interval, performance_field.key] = round(target_total_duration / total_batch_duration, 4)

```

```

# --- Quality ---
batch_quality_losses = pd.to_timedelta(batches[batch_quality_field.key], unit='m')
total_quality_loss = (in_month_ratio * batch_quality_losses).sum(skipna=False)
months.loc[interval, quality_field.key] = round(1 - total_quality_loss / target_total_duration, 4)

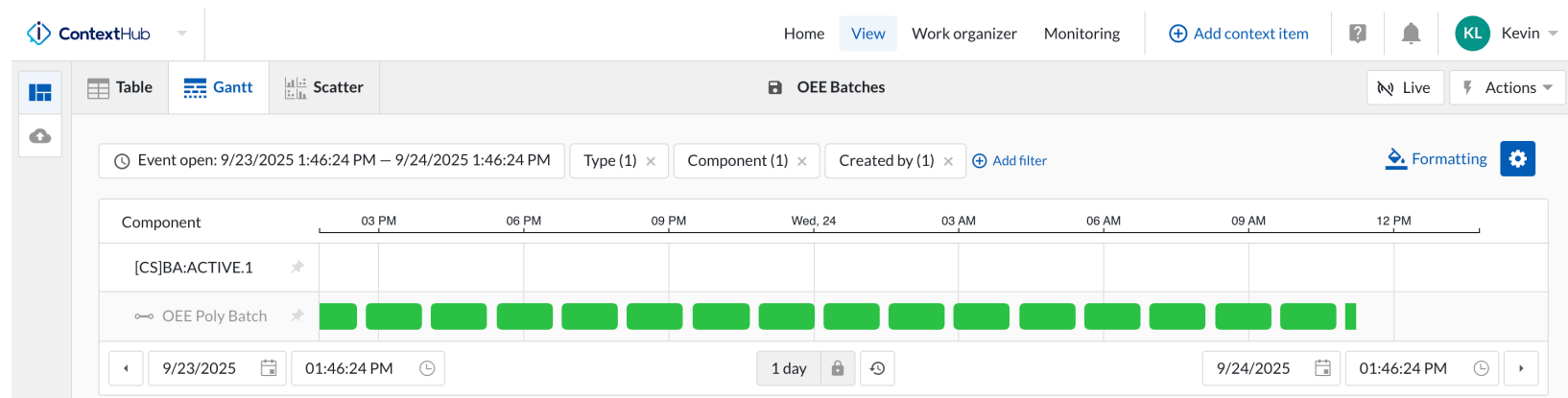
# --- Overall OEE (A x P x Q) ---
months.loc[interval, oee_total_field.key] = round((target_total_duration - total_quality_loss) / inter

# Write back OEE results for each month to ContextHub
months.context.update()

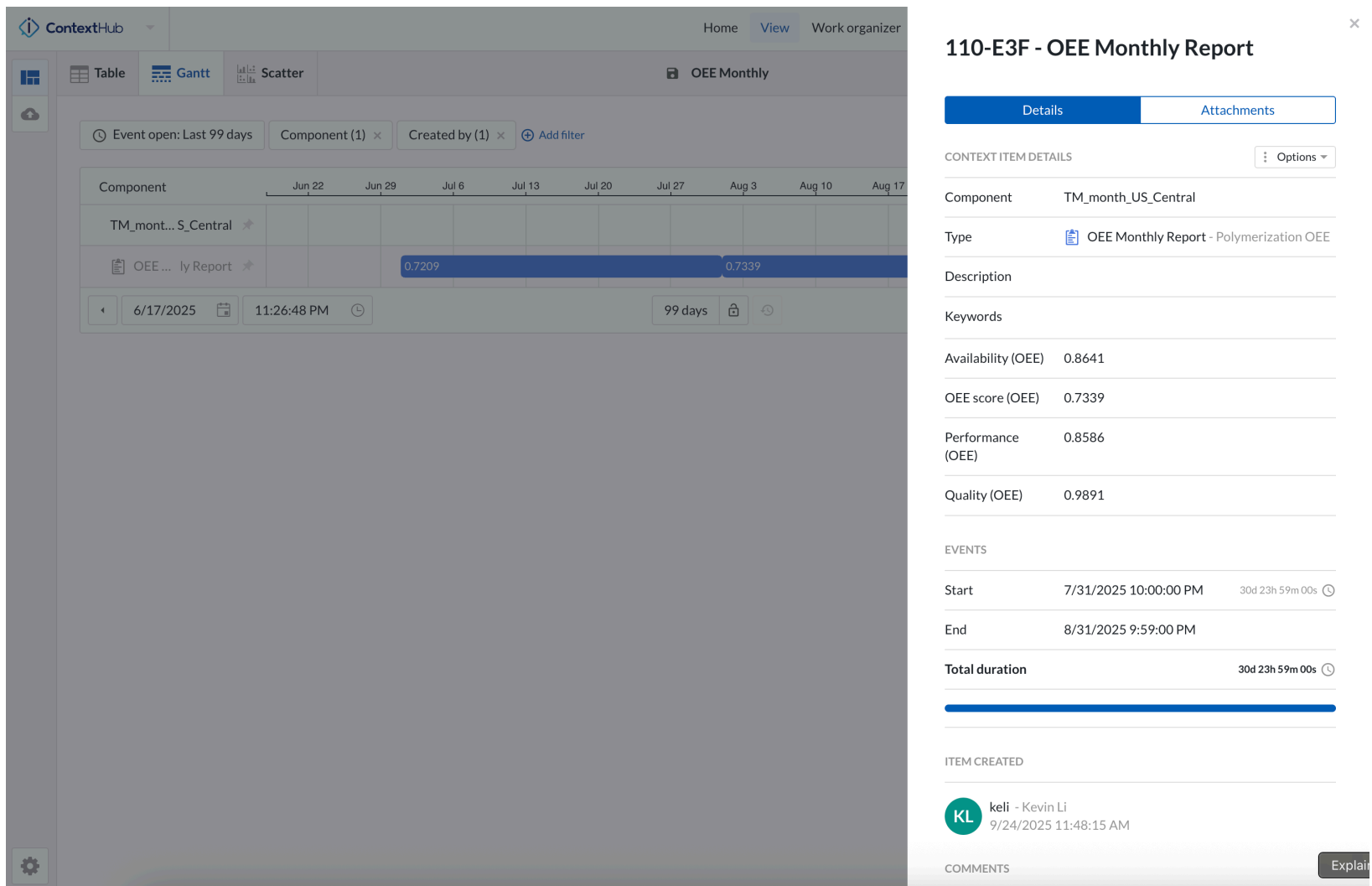
```

Visualize and validate results (ContextHub)

Create context views and Gantt charts to visualize the updated context items. Verify that the results match expectations.



Apply context item filters and view context items in Gantt format.



Verify the calculated field values in the reporting period context items.

Note: Before moving on to the next step (creating custom visualizations in MLHub), save your context views. We will be referencing these views in the MLHub notebook.

Create custom visualizations (MLHub)

MLHub can be used to create custom visualizations. We can load ContextHub views into MLHub and extract the OEE metrics from the views. The metrics can then be deployed to a custom visualization. For example, we can create multi-line charts, bar charts, and radar plots of the OEE metrics in MLHub. Then we create a pipeline of the visualization to display on a DashHub dashboard.

Steps to Create a Custom Visualization

1. **Open MLHub** and start a new notebook.

2. **Add TrendMiner content:**

- Click the blue **“+”** button.
- Select the **Context View** that contains your context items and OEE metrics (from the previous step).

3. **Initialize the notebook:**

- Click the **initialization script**. This inserts a code cell with the necessary imports and client setup.
- Then click your **Context View** under TrendMiner content. This automatically adds a code cell that loads the context view into a pandas DataFrame.

The screenshot shows the MLHub interface. On the left, a sidebar lists notebooks under 'TRENDMINER CONTENT' and 'SNIPPETS'. The main area displays the 'OEE Solution Notebook' with two code blocks. The first block (labeled [1]) contains suggested data science package imports (pandas, numpy, datetime, matplotlib) and the TrendMiner package import. The second block (labeled [2]) contains code for loading the ContextHub view, including creating a ContextView object and loading the 'OEE Monthly' view.

```
[1]: # Suggested data science package imports
import pandas as pd
import numpy as np

from datetime import datetime
import matplotlib
import matplotlib.pyplot as plt

# TrendMiner package import
import trendminer
from trendminer.trendminer_client import TrendMinerClient

import os

token = os.environ["KERNEL_USER_TOKEN"]

# Create TrendMiner API object
client = TrendMinerClient(token)

[2]: # Loading ContextHub view: OEE Monthly
from trendminer.context.context_view import ContextView
import uuid

context_view = ContextView(client, uuid.UUID("41df698f-2d29-4b67-9c88-c3d99c59ca95"))
df_month = context_view.load_view()
```

MLHub notebook: initialization script & load context view code blocks added (after completing **Step 3**).

4. Build visualizations:

- With your OEE data now in a DataFrame, you can create custom plots.
- Examples include:
 - Multi-line chart (OEE vs Availability, Performance, Quality)
 - Stacked bar chart (loss contributions per component)
 - Radar plot (Availability, Performance, Quality for recent months)

Tip: there are many visualization libraries in Python. Some common ones are 'matplotlib', 'plotly', and 'seaborn'. Using these libraries, you can create visualizations such as gauge plots, heatmaps, spider plots, etc. all within MLHub.

```

In [ ]: # Provided is an example visualization - this code creates a radar plot!
# --- RADAR: A/P/Q for the last N months ---

import plotly.graph_objects as go

LAST_N = 4 # change to show more/fewer months

dfr = df_month.copy()
dfr["start_date"] = pd.to_datetime(dfr["start_date"])
dfr = dfr.sort_values("start_date").tail(LAST_N)

# Build a radar (polar) chart with one trace per month
categories = ["OEE_availability", "OEE_performance", "OEE_quality"]
cat_labels = ["Availability", "Performance", "Quality"]

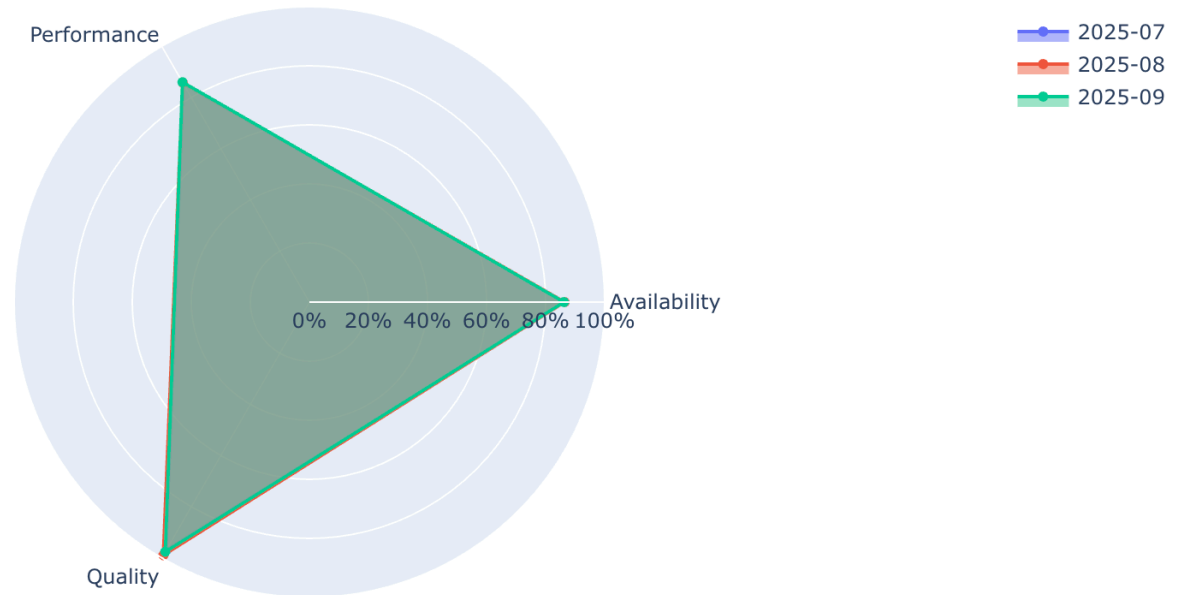
fig = go.Figure()

for _, row in dfr.iterrows():
    values = [row[c] for c in categories]
    # Close the polygon by repeating the first point
    values_closed = values + values[:1]
    fig.add_trace(go.Scatterpolar(
        r=values_closed,
        theta=cat_labels + cat_labels[:1],
        name=row["start_date"].strftime("%Y-%m"),
        mode="lines+markers",
        fill="toself",
    ))

fig.update_layout(
    title=f"Radar of A/P/Q for Last {len(dfr)} Months",
    polar=dict(
        radialaxis=dict(range=[0, 1], tickformat=".0%", showline=True),
    ),
    showlegend=True,
)
fig.show()

```

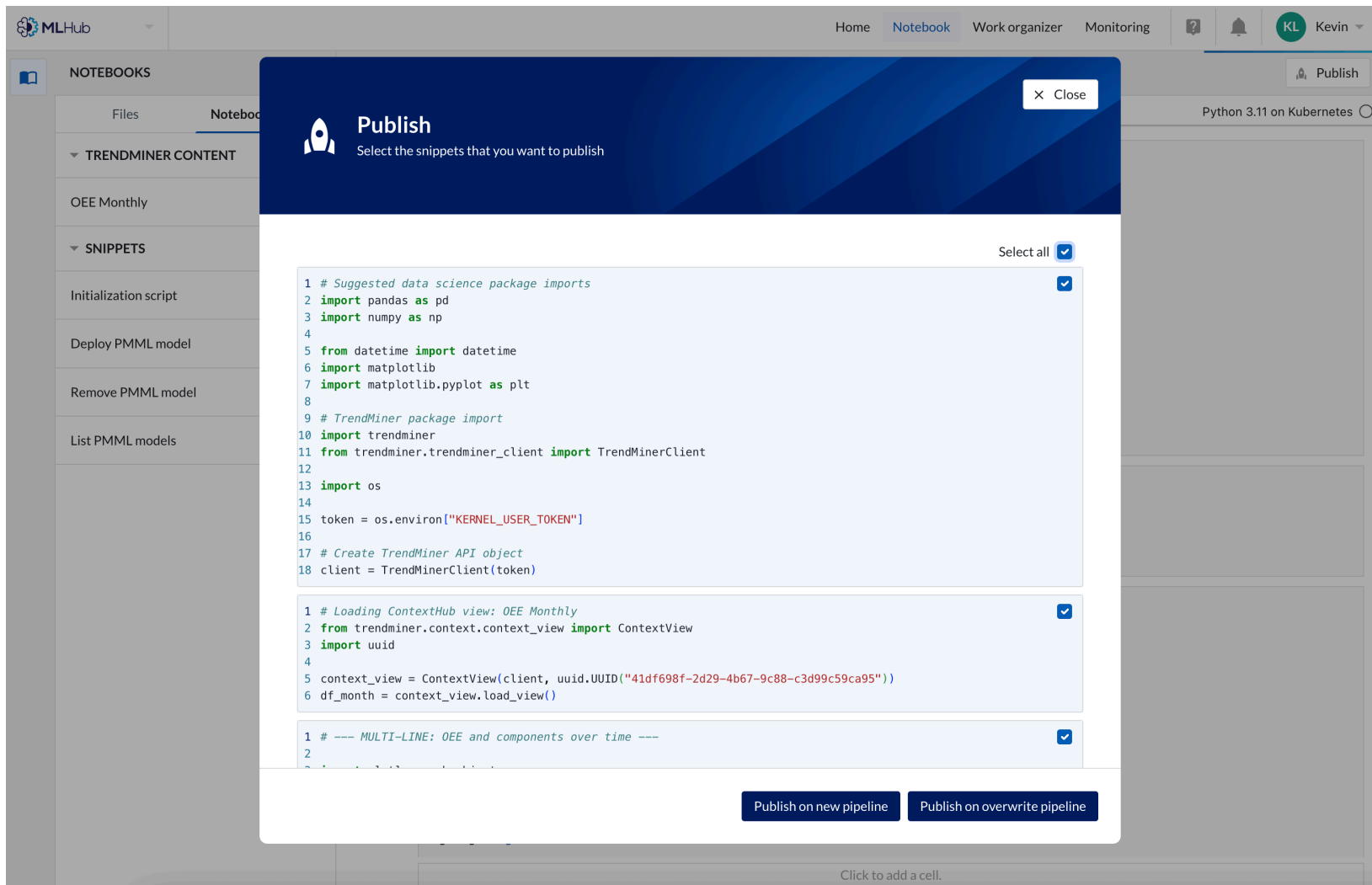
Radar of A/P/Q for Last 3 Months



Example custom visualization: radar plot of the A/P/Q OEE metrics from the last 3 months.

5. Publish to DashHub:

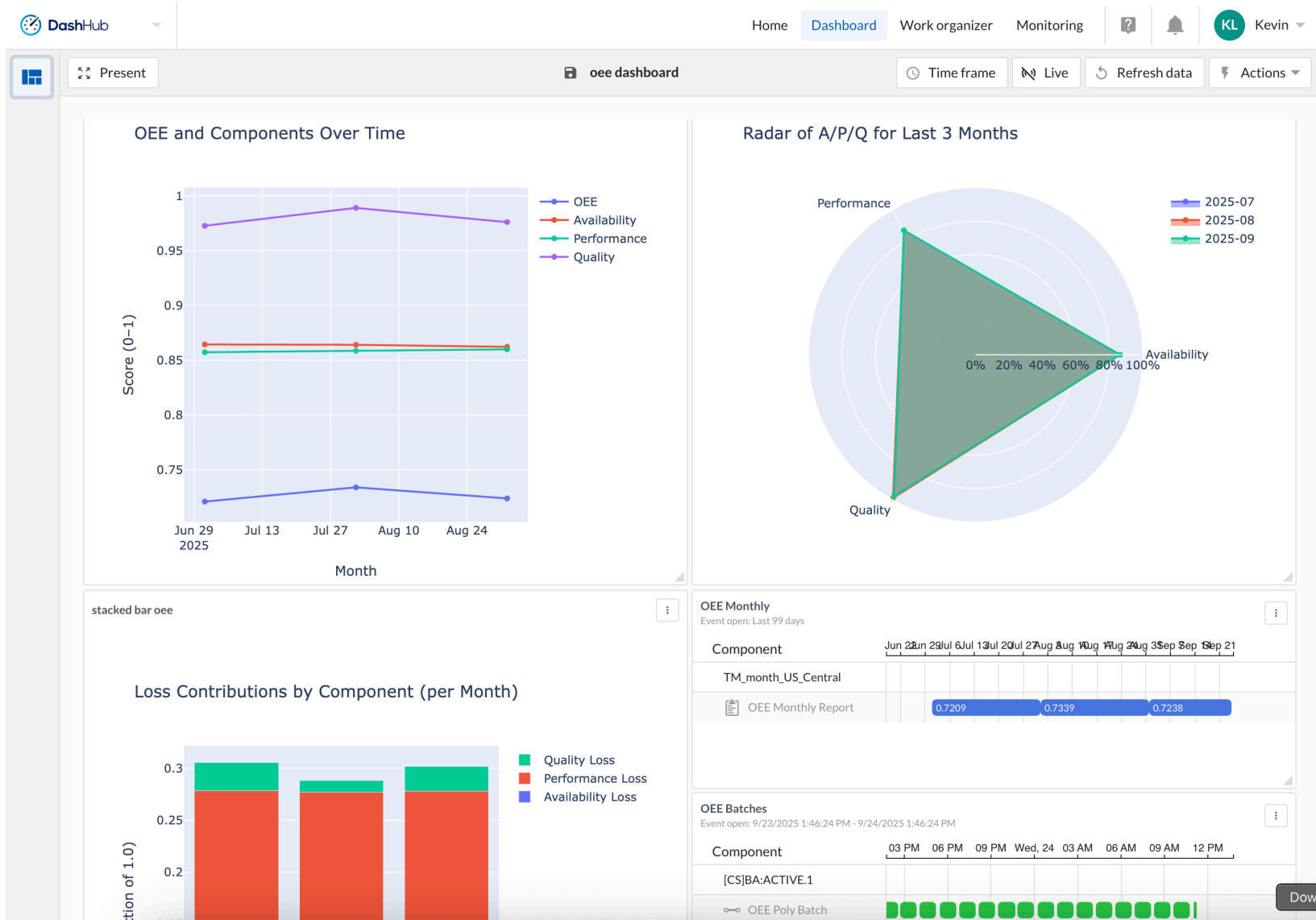
- When the visualizations are ready, click **Publish** (top-right).
- Select all cells required for your visualization.
- Save the pipeline — your visualizations are now ready to be deployed in **DashHub**.



Publish and save pipeline. The pipeline is needed for the 'Notebook output' tile in DashHub.

Build dashboard (DashHub)

Navigate to DashHub. Custom visualizations created in MLHub can be added as a tile via Notebook output tile after choosing the corresponding pipeline.



Dashboard with several custom visualizations embedded (multi-line chart, stacked bar, radar plot).